Aim: To create a simple cryptocurrency using Python by implementing blockchain technology and to perform mining using the Proof-of-Work consensus mechanism.

Problem Statement: To develop a basic cryptocurrency system that maintains transactions in a blockchain, generates blocks using cryptographic hashing, performs mining using Proof-of-Work, provides mining rewards, and verifies the integrity of the blockchain.

Technologies Used

- **Programming Language:** Python
- **Platform:** Google Colab
- **Hashing Algorithm:** SHA-256
- **Consensus Mechanism:** Proof-of-Work
- **Library:** hashlib
- **Data Structure:** Python Classes, Lists and Dictionaries

Theory: Cryptocurrency is a digital currency that uses cryptography to secure transactions. Blockchain is the underlying technology that stores these transactions in a sequence of connected blocks. Each block contains transaction data, a timestamp, and a unique hash that connects it to the previous block.

In this experiment, a basic cryptocurrency is implemented using Python. The SHA-256 algorithm is used for hashing, and the Proof-of-Work mechanism is used for mining. During mining, a suitable nonce is found to generate a hash that satisfies the given difficulty level. The mined block is then added to the blockchain, and the miner receives a reward. Finally, the blockchain is validated to ensure data integrity.

Step 1 — Import Required Libraries

```
import hashlib

import json

import time
```

Step 2 — Create the Block Class

```
class Block:
    def __init__(self, index, transactions, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_data = {
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }

        block_string = json.dumps(block_data, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()
```

Step 3 — Create the Blockchain Class

```
lass Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.pending_transactions = []

    def create_genesis_block(self):
        return Block(0, [], "0")
```

Step 4 — Add the Mining Function

```
def mine_block(self, block, difficulty):
    target = "0" * difficulty

    while not block.hash.startswith(target):
        block.nonce += 1
        block.hash = block.calculate_hash()

    return block
Blockchain.mine_block = mine_block

print("Mining function added successfully!")
```

```
Mining function added successfully!
```

Step 5 — Add Transactions

```
def add_transaction(self, sender, receiver, amount):
    transaction = {
        "sender": sender,
        "receiver": receiver,
        "amount": amount
    }

    self.pending_transactions.append(transaction)

Blockchain.add_transaction = add_transaction
```

```
print("Transaction function added successfully!")
```

```
Transaction function added successfully!
```

Step 6: **create the DeepCoin blockchain and add transactions**.

```
# Create the DeepCoin blockchain
deepcoin = Blockchain()

# Add sample transactions
deepcoin.add_transaction("deepak", "advani", 20)
deepcoin.add_transaction("deep", "adani", 10)

print("Cryptocurrency: DeepCoin")
print("Pending Transactions:")
print(deepcoin.pending_transactions)
```

```
Cryptocurrency: DeepCoin
Pending Transactions:
[{'sender': 'deepak', 'receiver': 'advani', 'amount': 20}, {'sender': 'deep', 'receiver'
```

Step 7 — Mine the Block

```
difficulty = 4

new_block = Block(
    len(deepcoin.chain),
    deepcoin.pending_transactions,
    deepcoin.chain[-1].hash
)

print("Mining started...")

deepcoin.mine_block(new_block, difficulty)

deepcoin.chain.append(new_block)
deepcoin.pending_transactions = []

print("\nBlock successfully mined!")
print("Block Index:", new_block.index)
print("Nonce:", new_block.nonce)
print("Hash:", new_block.hash)
```

```
Mining started...

Block successfully mined!
Block Index: 1
Nonce: 143782
Hash: 00003fc05f9c117dadd4925bc4ed7088a7b47d37da1e81de099262c5d03868d6
```

Step 8 — Add Mining Reward

```
# Mining reward
miner = "Miner1"
reward = 10

reward_transaction = {
   "sender": "Network",
   "receiver": miner,
   "amount": reward
}

deepcoin.chain[-1].transactions.append(reward_transaction)

# Recalculate the block hash
deepcoin.chain[-1].hash = deepcoin.chain[-1].calculate_hash()

print("Mining Reward Added!")
print("Miner:", miner)
print("Reward:", reward, "DeepCoins")
```

```
Mining Reward Added!
Miner: Miner1
Reward: 10 DeepCoins
```

Step 9 — Display the Blockchain

```
print("========== DEEPCOIN BLOCKCHAIN ==========")

for block in deepcoin.chain:
   print("\nBlock Index:", block.index)
   print("Timestamp:", block.timestamp)
   print("Transactions:", block.transactions)
   print("Previous Hash:", block.previous_hash)
   print("Nonce:", block.nonce)
   print("Hash:", block.hash)
```

```
print("\n========== END OF BLOCKCHAIN ==========")
```

```
========== DEEPCOIN BLOCKCHAIN ==========

Block Index: 0
Timestamp: 1788333750.1474154
Transactions: []
Previous Hash: 0
Nonce: 0
Hash: 38a2b2e672c189c8b7c08c78a7af562d65571272c083d56e2a4f677677a4432e

Block Index: 1
Timestamp: 1788334143.3866992
Transactions: [{'sender': 'deepak', 'receiver': 'advani', 'amount': 20}, {'sender': 'dee
Previous Hash: 38a2b2e672c189c8b7c08c78a7af562d65571272c083d56e2a4f677677a4432e
Nonce: 143782
Hash: b45bff78dd59d498ece161a7c9bc8810374c336ea7ceffcc28f0359755da3d36

========== END OF BLOCKCHAIN ==========
```

Step 10 — Validate the Blockchain

```
def is_chain_valid(blockchain):
    for i in range(1, len(blockchain.chain)):
        current_block = blockchain.chain[i]
        previous_block = blockchain.chain[i - 1]

        if current_block.hash != current_block.calculate_hash():
            return False

        if current_block.previous_hash != previous_block.hash:
            return False

    return True


print("Is the blockchain valid?", is_chain_valid(deepcoin))
```

```
Is the blockchain valid? True
```

Step 11 — Display Final Blockchain Status

```
print("========== FINAL DEEPCOIN STATUS ==========")
print("Cryptocurrency: DeepCoin")
print("Total Blocks:", len(deepcoin.chain))
print("Pending Transactions:", len(deepcoin.pending_transactions))
print("Blockchain Valid:", is_chain_valid(deepcoin))
print("Mining Reward: 10 DeepCoins")
print("========================================")
```

```
========== FINAL DEEPCOIN STATUS ==========
Cryptocurrency: DeepCoin
Total Blocks: 2
Pending Transactions: 0
Blockchain Valid: True
Mining Reward: 10 DeepCoins
========================================
```

Step 12 — Calculate Cryptocurrency Balances

```
balances = {}

for block in deepcoin.chain:
    for transaction in block.transactions:
        sender = transaction["sender"]
        receiver = transaction["receiver"]
        amount = transaction["amount"]

        if sender != "Network":
            balances[sender] = balances.get(sender, 0) - amount

        balances[receiver] = balances.get(receiver, 0) + amount

print("========== DEEPCOIN BALANCES ==========")

for user, balance in balances.items():
    print(user, ":", balance, "DeepCoins")
```

```
========== DEEPCOIN BALANCES ==========
deepak : -20 DeepCoins
advani : 20 DeepCoins
deep : -10 DeepCoins
adani : 10 DeepCoins
Miner1 : 10 DeepCoins
```

Step 13 — Final Verification

```
print("========== FINAL VERIFICATION ==========")
print("Cryptocurrency : DeepCoin")
print("Total Blocks   :", len(deepcoin.chain))
print("Last Block Hash:", deepcoin.chain[-1].hash)
print("Blockchain Valid:", is_chain_valid(deepcoin))
print("Mining Status  : Successful")
print("=======================================")
```

```
========== FINAL VERIFICATION ==========
Cryptocurrency : DeepCoin
Total Blocks   : 2
Last Block Hash: b45bff78dd59d498ece161a7c9bc8810374c336ea7ceffcc28f0359755da3d36
Blockchain Valid: True
Mining Status  : Successful
=======================================
```

Conclusion: The experiment demonstrated the basic working of a cryptocurrency and blockchain. Transactions were securely stored in blocks, mining was performed using Proof-of-Work, and blockchain integrity was verified successfully.